

Guía de GitHub para el equipo

Guía de GitHub para el equipo Triverde

Esta guía te lleva de cero a trabajar en el repositorio sin romper nada. No necesitas saber Git de antes: sigue los pasos en orden. Si algo no calza, pregunta antes de forzar — en Git casi todo se puede deshacer, pero es más fácil no meter la pata.

La regla de oro: nadie trabaja directo sobre `main`. Cada cambio entra por una rama y un Pull Request que un compañero revisa. Así `main` siempre queda funcionando.

1. Preparación (una sola vez)

1.1 Instalar Git

- **Windows:** descarga *Git for Windows* desde git-scm.com e instálalo con las opciones por defecto.
- Comprueba que quedó instalado abriendo una terminal y escribiendo:

```
git --version
```

1.2 Configurar tu identidad — ¡ojo con esto!

Cada commit lleva tu nombre y correo. Para que tus commits **cuenten** en el informe de aportes del proyecto (lo revisa el profesor), el correo que configures aquí **tiene que ser un correo verificado en tu cuenta de GitHub**. Si no, tus commits aparecen como de un desconocido y no se te atribuyen.

```
git config --global user.name "Tu Nombre"
git config --global user.email "el-correo-de-tu-cuenta-github@ejemplo.com"
```

Para ver qué correos tienes verificados: en GitHub, *Settings* → *Emails*. Usa uno de esos.

1.3 Aceptar la invitación al repositorio

Kenji te va a invitar por correo como colaborador. Abre el correo (o ve a github.com/notifications) y acepta la invitación. Sin esto no puedes subir cambios.

2. Clonar el repositorio (una sola vez)

Clonar = bajar una copia del proyecto a tu computador.

```
git clone <URL-del-repositorio>
```

La URL te la pasa Kenji (botón verde **Code** en GitHub → HTTPS). Esto crea una carpeta con todo el proyecto. Entra en ella:

```
cd Triverde-repo
```

La primera vez que subas algo, GitHub te pedirá iniciar sesión: se abre una ventana del navegador, autorizas y listo (no tienes que escribir contraseña en la terminal).

3. El ciclo de trabajo diario

Cada vez que vayas a trabajar, repite este ciclo. Son cinco pasos.

Paso 1 — Traer lo último

Antes de empezar, sincroniza tu copia con lo que el equipo ya subió:

```
git switch main  
git pull
```

Paso 2 — Crear tu rama

Nunca trabajes sobre `main`. Crea una rama con un nombre que diga qué vas a hacer:

```
git switch -c diagrama-casos-de-uso-modulo-4
```

(Usa guiones, sin espacios ni tildes. Ejemplos: `ishikawa`, `cv-juan`, `bpmn-as-is`.)

Paso 3 — Trabajar y guardar (commit)

Haz tu trabajo (edita archivos, agrega tu diagrama, etc.). Cuando tengas un avance con sentido, guárdalo:

```
git add .  
git commit -m "Agrego diagrama de casos de uso del Módulo 4"
```

- `git add .` marca todos tus cambios para guardar.
- `git commit -m "..."` los guarda con un mensaje que explica **qué** hiciste.
- Puedes hacer **varios commits** en una misma rama, uno por cada avance. Es sano commitear seguido.

Paso 4 — Subir tu rama (push)

```
git push -u origin diagrama-casos-de-uso-modulo-4
```

(La primera vez usa `-u origin <nombre-rama>`; después basta `git push .`)

Paso 5 — Abrir el Pull Request (PR)

El PR es la propuesta de meter tu rama a `main` para que un compañero la revise.

1. Ve al repositorio en GitHub. Aparece un aviso amarillo con tu rama y un botón **Compare & pull request**. Presiónalo.
2. Escribe un título y una descripción corta de qué hiciste.
3. Crea el PR. Avisa al equipo para que alguien lo revise.
4. Si el revisor pide cambios, los haces en la **misma rama** (repites Paso 3 y 4); el PR se actualiza solo.
5. Cuando esté aprobado, se hace **merge** (lo hace el revisor o quien Kenji defina). Tu trabajo ya está en `main`.

4. Revisar el PR de un compañero

Cuando te toque revisar:

1. Entra al PR en GitHub, pestaña **Files changed**: ahí ves todo lo que cambió.
2. Si algo se puede mejorar, deja un comentario en la línea (**Request changes**). Si está bien, apruébalo (**Approve**).
3. Sé concreto y amable: “faltó nombrar el archivo por módulo” es mejor que “está malo”.

5. Chuleta de comandos

Quiero...	Comando
Ver en qué rama estoy y qué cambié	<code>git status</code>
Traer lo último de <code>main</code>	<code>git switch main</code> y <code>git pull</code>
Crear una rama nueva	<code>git switch -c nombre-rama</code>
Cambiarme a una rama que ya existe	<code>git switch nombre-rama</code>
Guardar mis cambios	<code>git add .</code> y <code>git commit -m "mensaje"</code>
Subir mi rama	<code>git push</code>
Ver el historial	<code>git log --oneline</code>

6. Errores comunes y qué hacer

- **“Trabajé sobre `main` sin querer.”** No entres en pánico, no hagas push. Avisa antes de subir; se puede mover el trabajo a una rama sin perderlo.
- **“Me sale conflicto al hacer pull o merge.”** Es normal cuando dos personas tocan lo mismo. No lo fuerces: avisa y lo resolvemos juntos la primera vez.
- **“Subí un archivo enorme o que no iba.”** Avisa antes de seguir; sacarlo después es más engorroso.
- **Duda cualquiera:** pregunta en el grupo. Es preferible una pregunta a un `main` roto.